

Arbitrary Precision Library

Kuruva Jeshwanth Krishna

May 2 2025

1 Abstract

This report outlines the implementation and usage of `aarithmetic.jar` library, which supports arbitrary precision for integer operations and 30 decimal precision for float operations. This report describes Design, implementations, provides class diagram in UML format, and includes to help users understand the entire structure of the library.

2 Introduction

This library is developed in order to achieve very high precision which is not possible to obtain from the data types like integer, float, double, etc. data types of Java. This level of precision is achieved through storing the numbers as strings and performing the basic arithmetic on the parts of the string to achieve great precision at the cost of Time complexity.

3 Design

This section describes the internal design and implementation structure of the library class. My approach focuses on the ability to handle arbitrarily large integers through manual arithmetic.

3.1 Structure of Implementation

I implemented a two public class `AInteger` and `AFloat` that includes:

- A Public String to store the number in base 10.
- A default constructor to initialize an integer to "0" and float to "0.0"
- A parametric constructor to initialize with the given string
- A parse method to create an object from a string

3.2 Methods

- Public static methods at a very high level:
 - `Addition()`
 - `Subtraction()`
 - `Multiplication()`
 - `Division()`
- Public static methods at slightly lower level
 - These functions are the helpers of the high level functions operating on strings

- `Add()`
 - `Subtract()`
 - `Multiply()`
 - `Divide()`
- More Public helper methods which are used at further lower level will be displayed later in the report

3.3 Design Decisions

There were some key decisions I made to ensure readability reusability and modularity of the code:

- **Helper functions:** This code has a lot of helper functions, This code deals more with string attribute of an object more than an object itself so I added the helper functions before hand for all the major calculations with string arguments
- **Modularity:** Each arithmetic operation is broken into smaller methods for clarity and easier debugging. Helper methods handle edge cases like normalization and magnitude comparison.
- **Single-Class Design:** All logic is encapsulated in a single class to keep the interface simple and focused, while still being maintainable.

4 Implementations

The key parts of the implementation of the classes will be presented in the following section

4.1 AInteger

A breif Implementation of AInteger's methods are provided below

- **Public AInteger():** Default constructor used to create an instance and initialize it to "0"
- **Public AInteger(String):** Parametric constructor used to create an instance and initialize it to the string provided in the argument
- **Public AInteger(AInteger):** Copy constructor this initializes the value of the object with the object provided in the argument
- **Public static AInteger parse :** Function to create an instance of AInteger from a string
- **Public static String add_strings :** This function adds two non-negative integers represented as strings, assuming the first string `string1` is not shorter than the second `string2`. It simulates manual digit-by-digit addition from right to left, maintaining a carry throughout the process.

At each step:

- Digits from both strings are added along with any carry.
- If `string2` is exhausted, only `string1` digits and carry are used.
- The resulting digit is prepended to the result string.

After the loop, if a carry remains, it is added to the front of the result. The final string represents the integer sum.

- **Public static String subtract_strings** : This function subtracts two non-negative integers represented as strings, assuming that `string1` is not shorter and not smaller than `string2`. It simulates manual digit-by-digit subtraction from right to left, maintaining a borrow when necessary.

At each step:

- Digits from both strings are subtracted, considering any existing borrow.
- If the digit in `string1` is smaller than the corresponding digit in `string2` plus the borrow, it borrows from the next higher digit.
- If `string2` is exhausted, only the digits of `string1` and borrow are processed.
- The resulting digit is prepended to the result string.

- **Public static Boolean isgreater** : This function compares two integer strings `string1` and `string2`, which may include negative signs, and returns `true` if `string1` is numerically greater than `string2`, otherwise `false`. It first removes any leading zeroes and handles sign comparison.

The logic proceeds as follows:

- If either string starts with a minus sign, their signs are evaluated to determine the result directly.
- If both numbers are negative, the result is determined by checking which absolute value is smaller.
- If signs are the same and both numbers are non-negative, the function compares lengths of the strings to decide.
- If lengths are equal, it compares digits from left to right to determine which string represents the greater value.

- **Public static Boolean islesser** : This function compares two integer strings `string1` and `string2`, which may include negative signs, and returns `true` if `string1` is numerically less than `string2`, otherwise `false`. Leading zeroes are removed before performing the comparison.

The logic proceeds as follows:

- If either string has a negative sign, their signs are handled to directly infer the result.
- If both are negative, the function uses the reversed logic by calling `isgreater` on their absolute values.
- If both numbers are non-negative, the lengths of the strings are compared.
- If lengths are equal, it compares digits from left to right to determine the lesser value.

- **Public static Boolean isgreater_or_equals** : This function compares two integer strings `string1` and `string2` and returns `true` if `string1` is greater than or equal to `string2`, otherwise `false`.

The logic is as follows:

- It returns `true` if `string1` is numerically greater than `string2`, determined using the `isgreater` function.
- If the strings are numerically equal after removing leading zeroes, it also returns `true`.
- Otherwise, it returns `false`.

- **Public static Boolean islesser_or_equals** : This function compares two integer strings `string1` and `string2` and returns `true` if `string1` is less than or equal to `string2`, otherwise `false`.

The logic is as follows:

- It returns `true` if `string1` is numerically less than `string2`, determined using the `islesser` function.
- If the strings are numerically equal after removing leading zeroes, it also returns `true`.
- Otherwise, it returns `false`.

- **Public static String remove_leading_zeroes :** This function removes any leading zeroes from a given integer string `string`, while preserving the sign if the number is negative.

The logic is as follows:

- It iterates through the string until it finds a non-zero digit (excluding any negative sign).
- If the string represents a negative number, the minus sign is retained and the substring starting from the first non-zero digit is returned.
- If all characters are zeroes, the function returns "0".

- **Public static String absolute_string :** This function returns the absolute value of an integer represented as a string by removing the negative sign if present.

The logic is as follows:

- If the first character of the string is a minus sign, it returns the substring excluding the sign.
- Otherwise, it returns the original string unchanged.

- **Public static String Addition :** This function adds two integer strings `string1` and `string2`, considering their signs and handling various edge cases. It uses the helper functions `add_strings` and `subtract_strings` to perform the addition after ensuring that the assumptions of these functions are met.

The logic proceeds as follows:

- First, leading zeroes are removed from both strings using the `remove_leading_zeroes` function.
- If both strings are negative:
 - * The absolute values of both strings are calculated.
 - * The strings are compared, and the larger absolute value is passed to the `add_strings` function to perform the addition.
 - * The result is prefixed with a negative sign.
- If exactly one string is negative:
 - * The absolute values of both strings are calculated.
 - * The larger absolute value is subtracted from the smaller one using the `subtract_strings` function, and the result is prefixed with the sign of the larger number.
- If both strings are positive:
 - * The strings are compared, and the addition is performed using `add_strings` based on the comparison.

- **Public static String Subtraction :** This function subtracts two integer strings `string1` and `string2` by using the `Addition` function. It calculates the subtraction by adding the first integer to the additive inverse of the second integer.

The logic is as follows:

- If `string2` is positive, it is converted to its negative form by prepending a minus sign.
- The `Addition` function is then called with `string1` and the modified `string2`.
- The result from the `Addition` function is returned as the subtraction result.

- **Public static String Multiplication :** This function multiplies two integer strings `string1` and `string2` by using repeated addition. The function first computes the product of the absolute values of the two strings and then prepends the appropriate sign to the result.

The logic is as follows:

- The absolute values of both input strings are obtained by removing leading zeroes and signs using the `absolute_string` and `remove_leading_zeroes` functions.
 - A check is performed to immediately return "0" if either of the input strings is "0".
 - A loop iterates over the digits of the second string, multiplying the first string by each digit of the second string (digit-by-digit multiplication).
 - For each multiplication, the result is added to a cumulative product using the `Addition` function, with appropriate zeroes padded based on the place of the digit in the second string.
 - After the multiplication is complete, the function prepends the appropriate sign based on the signs of the input strings.
- **Public static String Division :** This function performs integer division of two integer strings `string1` and `string2` by using repeated subtraction. It computes the quotient by subtracting the divisor from the dividend until the remainder is smaller than the divisor.

The logic is as follows:

- The absolute values of both input strings are obtained by removing leading zeroes and signs using the `absolute_string` and `remove_leading_zeroes` functions.
 - An exception is thrown if the divisor is zero to avoid division by zero.
 - If the dividend is smaller than the divisor, the quotient is immediately set to "0".
 - A loop iterates through the digits of the dividend, creating a temporary dividend and performing repeated subtraction of the divisor from it.
 - The quotient is built by counting how many times the divisor can be subtracted from the temporary dividend.
 - After the division is complete, the function removes any leading zeroes from the quotient and prepends the appropriate sign based on the signs of the input strings.
- **Public static AInteger Add :** This function passes the string attribute to its respective helper parses it into an object and returns the answer
 - **Public static AInteger Subtract :** This function passes the string attribute to its respective helper parses it into an object and returns the answer
 - **Public static AInteger Multiply :** This function passes the string attribute to its respective helper parses it into an object and returns the answer
 - **Public static AInteger Divide:** This function passes the string attribute to its respective helper parses it into an object and returns the answer

4.2 AFloat

- A brief Implementation of AFloat's methods are provided below
 - **Public AFloat():** Default constructor used to create an instance and initialize it to "0.0"
 - **Public AFloat(String):** Parametric constructor used to create an instance and initialize it to the string provided in the argument
 - **Public AFloat(AFloat):** Copy constructor this initializes the value of the object with the object provided in the argument
 - **Public static AFloat parse :** Function to create an instance of AInteger from a string

- **Public static Boolean isdecimal :** This function checks whether the input string `string` represents a decimal number by verifying the presence of a decimal point.

The logic is as follows:

- The function iterates over each character in the input string.
- If any character is a '.', the function returns `true`, indicating that the number is a decimal.
- If no decimal point is found after examining the entire string, the function returns `false`.

- **Public static String remove_leading_zeroes :** This function removes the leading zeroes from the given string `string` while preserving its sign.

The logic is as follows:

- The function first checks if the input string is a decimal by calling the `isdecimal` method.
- If the string is not a decimal (i.e., an integer), it delegates the removal of leading zeroes to the `AInteger.remove_leading_zeroes` method.
- For decimal strings, the function iterates through the characters of the string to find the first non-zero character or the decimal point.
- If the string contains a decimal point and all characters before it are zeroes, it removes these zeroes and ensures the sign of the string is preserved.
- The function returns the string with leading zeroes removed, and the sign preserved (if present).

- **Public static String remove_trailing_zeroes :** This function removes all the trailing zeroes after the decimal point from the given string `string` and returns the modified string.

The logic is as follows:

- The function first checks if the string is a decimal by calling the `isdecimal` method.
- If the string is a decimal, the function starts from the last character and iterates backward to find the last non-zero character.
- If the last non-zero character is a decimal point, the function ensures that the decimal point is preserved and removes the trailing zeroes.
- If there are no trailing zeroes, or the string is an integer, the function returns the string as is.
- For integers, the string is returned without any modifications since trailing zeroes do not exist in integers.

- **Public static String remove_both_leading_and_trailing_zeroes :** This function removes both the leading and trailing zeroes from the given string `string`.

The logic is as follows:

- The function first removes the leading zeroes by calling the `remove_leading_zeroes` method.
- After removing the leading zeroes, it then removes the trailing zeroes by calling the `remove_trailing_zeroes` method.
- This ensures that the string is left with only significant digits, with no unnecessary zeroes at either end.

- **Public static int decimal_places :** This function counts the number of elements (digits) present after the decimal point in the given string `string`.

The logic is as follows:

- The function first checks if the string contains a decimal point by calling the `isdecimal` function.
 - If the string contains a decimal point, the function iterates backwards through the string until it finds the decimal point.
 - It then calculates the number of digits that appear after the decimal point by subtracting the position of the decimal point from the total length of the string.
 - If the string does not contain a decimal point (i.e., it is an integer), the function returns 0.
- **Public static int decimal_index :** This function returns the index of the decimal point in the given string `string`.

The logic is as follows:

- The function calculates the number of decimal places in the string by calling the `decimal_places` function.
 - It then computes the index of the decimal point by subtracting the number of decimal places from the total length of the string and adjusting for the position of the decimal point.
- **Public static String Truncate :** This function ensures that the number of decimal places in the given string `string` is less than or equal to 30. If the string has more than 30 decimal places, it truncates the string to 30 decimal places.

The logic is as follows:

- The function first checks if the number of decimal places in the string is less than or equal to 30 using the `decimal_places` function.
 - If the number of decimal places is already less than or equal to 30, it returns the original string.
 - If the number of decimal places exceeds 30, it truncates the string to the first 30 decimal places using the `substring` function.
- **Public static String Addition :** This function returns the sum of two given strings, `string1` and `string2`, while handling both integer and decimal cases. It uses the `Addition` function from the `AInteger` class to compute the sum without the decimal point, then places the decimal point in the resulting sum.

The logic is as follows:

- If both strings are integers (i.e., they do not contain a decimal point), the function delegates the addition to the `AInteger.Addition` method.
- If either string is not a decimal, the function appends a ".0" to convert it into a decimal.
- Leading and trailing zeroes are removed from both strings using the `remove_both_leading_and_trailing_zeroes` function.
- The maximum number of decimal places between the two strings is computed to align the decimal points in the final result.
- Trailing zeroes are padded to ensure both strings have the same number of decimal places.
- The decimal points are removed from both strings to treat them as integers, and the sum is computed using the `AInteger.Addition` function.
- After the sum is computed, the decimal point is placed in the appropriate position, and the result is truncated to a maximum of 30 decimal places using the `Truncate` function.
- If the result is negative, the sign is preserved.

- **Public static String Subtraction :** This function returns the result of subtracting `string2` from `string1` by using the fact that subtraction is the addition of the first number and the additive inverse of the second number.

The logic is as follows:

- If `string2` is negative (i.e., it starts with a '-'), the negative sign is removed.
- If `string2` is positive, a negative sign is prepended to it, effectively making it the additive inverse.
- The `Addition` function is then called with `string1` and the modified `string2` (which has the opposite sign) to perform the addition.
- The result from the `Addition` function is returned as the result of the subtraction.

- **Public static String Multiplication :** This function returns the result of multiplying `string1` and `string2` by first converting them into integers and performing multiplication, then appropriately placing the decimal point in the result.

The logic is as follows:

- If neither of the input strings is decimal, the `AInteger.Multiplication` function is used to perform integer multiplication.
- If either of the strings is not decimal, a `.0` is appended to make it a decimal.
- Both strings are cleaned by removing leading and trailing zeroes using the `remove_both_leading_and_trailing_zeroes` function.
- The number of decimal places in both strings is calculated, and padding is added to ensure they have the same number of decimal places.
- The decimal point is removed from both strings to perform integer multiplication.
- The result of the multiplication is then calculated by calling the `AInteger.Multiplication` function.
- The result is formatted back by inserting the decimal point at the appropriate position, based on the sum of the decimal places of both strings.
- The final result is truncated to a precision of 30 decimal places and leading/trailing zeroes are removed.
- The sign of the result is adjusted based on the signs of the input strings. If both strings are positive or both negative, the result is positive; otherwise, it is negative.

- **Public static String Division :** This function returns the quotient of `string1` and `string2` by performing the division operation on the two decimal strings and then adjusting for decimal points and signs.//

The logic is as follows:

- If either of the input strings is not a decimal, a `.0` is appended to convert it into a decimal.
- The `remove_both_leading_and_trailing_zeroes` function is used to remove any unwanted zeroes from both strings.
- The number of decimal places in both input strings is calculated using the `decimal_places` function.
- The strings are padded with zeroes to ensure they have the same number of decimal places.
- The decimal point is removed from both strings to perform the division, and absolute values are used to avoid issues with signs.
- The `AInteger.Division` function is called to perform the integer division.
- The result is then formatted by placing the decimal point at the appropriate position based on the decimal places.

- If the result has fewer decimal places than required, additional zeroes are appended to the result.
- The final result is truncated to 30 decimal places using the **Truncate** function.
- The sign of the result is adjusted based on the signs of the input strings. If both strings are either positive or both negative, the result is positive; otherwise, it is negative.

5 UML DIAGRAMS

AInteger
Attributes
- integer : String
Constructors
+ AInteger() + AInteger(string : String) + AInteger(other : AInteger)
Static Methods
+ parse(string : String) : AInteger + Add(integer1 : AInteger, integer2 : AInteger) : AInteger + Subtract(integer1 : AInteger, integer2 : AInteger) : AInteger + Multiply(integer1 : AInteger, integer2 : AInteger) : AInteger + Divide(integer1 : AInteger, integer2 : AInteger) : AInteger + Addition(string1 : String, string2 : String) : String + Subtraction(string1 : String, string2 : String) : String + Multiplication(string1 : String, string2 : String) : String + Division(string1 : String, string2 : String) : String + add_strings(string1 : String, string2 : String) : String + subtract_strings(string1 : String, string2 : String) : String + isgreater(s1 : String, s2 : String) : Boolean + islesser(s1 : String, s2 : String) : Boolean + isgreater_or_equals(s1 : String, s2 : String) : Boolean + islesser_or_equals(s1 : String, s2 : String) : Boolean + remove_leading_zeroes(string : String) : String + absolute_string(string : String) : String

AFloat
Attributes
- number : String
Constructors
+ AFloat() + AFloat(string : String) + AFloat(other : AFloat)
Static Methods
+ parse(string : String) : AFloat + remove_leading_zeroes(string : String) : String + isdecimal(string : String) : Boolean + remove_trailing_zeroes(string : String) : String + remove_both_leading_and_trailing_zeroes(string : String) : String + decimal_places(string : String) : int + decimal_index(string : String) : int + Truncate(string : String) : String + Addition(string1 : String, string2 : String) : String + Subtraction(string1 : String, string2 : String) : String + Multiplication(string1 : String, string2 : String) : String + Division(string1 : String, string2 : String) : String + Add(number1 : AFloat, number2 : AFloat) : AFloat + Subtract(number1 : AFloat, number2 : AFloat) : AFloat + Multiply(number1 : AFloat, number2 : AFloat) : AFloat + Divide(number1 : AFloat, number2 : AFloat) : AFloat

6 Verification

This section provides some test cases which I ran in different ways

- MyInfArith :
 - java MyInfArith int add 789 324 = 1113
 - java MyInfArith int sub 789 3240 = -2451
 - java MyInfArith int mul 325 715 = 232375
 - java MyInfArith int div 346 21 = 17
- Project_runner :
 - This is the name of the python script I used.
 - python3 Project_runner.py float add 4.77 1.23 = 6
 - python3 Project_runner.py float sub -2.8 1.25 = -4.05
 - python3 Project_runner.py float mul -280000.234 1.256 = -351680.293904
 - python3 Project_runner.py float div -28 1.256 = -22.29299363057324840764331210191
- ant :
 - ant run -Dargs="int add 23650078224912949497310933240250 42939783262467113798386384401498"
= 66589861487380063295697317641748
 - ant run -Dargs="int sub 3116511674006599806495512758577 57745242300346381144446453884008"
= -54628730626339781337950941125431
 - ant run -Dargs="float mul 6400251.9377695 2326541.6827934" = 14890452913599.9717457253213

– ant run -Dargs="float div 3227 555" = 5.814414414414414414414414414414

These are some test cases which I verified.

7 Key learnings :

This is a list of things I learnt while making this project

- OOP concepts
- various functions in String class of java
- various git commands
- jar file creation
- python for execution scripts
- lot of latex aswell

8 Limitations :

These are some instances for which my code is not ready yet.

- Valid inputs are assumed
- Decimal precision is limited to 30
- This code doesnt roundoff it just truncates
- String length cant exceed the value of INT_MAX

9 Areas of Improvement :

- The precision of the AFloat can also be increased to any arbitrary value
- More functions like modulo and better comparators can be added
- Storing the string in the bases of 10000 instead of 10 can be done to optimise.
- Some other Data structures can be used to reduce time complexity.